

DBMS Course Notes (Chapters 1-5)

WEEK 1: Introduction to Databases and DBMS

What is a Database?

A structured collection of data that can be accessed, managed, and updated. Serves as a repository for organized data that supports efficient retrieval and management.

Examples: customer directory (e-commerce), hospital patient records, student database.

Characteristics:

1. Organized and structured
2. Persistent and stored electronically
3. Accessible by multiple users or applications

What is a DBMS?

Software designed to define, manipulate, retrieve, and manage data in a database.

Core Functions:

- Data Storage — stores large amounts of data efficiently
- Data Retrieval — quick access via queries
- Data Manipulation — insertion, deletion, updating
- Data Security — authorized access, data integrity

Popular DBMS Examples:

- Relational: MySQL, PostgreSQL, Oracle Database
- NoSQL: MongoDB, Cassandra, DynamoDB
- Cloud: Google Cloud Spanner, Amazon RDS

Data vs Database

Aspect	Data	Database
Definition	Raw, unprocessed facts	Organized collection of structured data
Example	"John, 25"	Table with columns: Name, Age
Purpose	Basic unit of information	Stores and manages related data

Components of a DBMS

- **Database Engine** — core component performing storage/retrieval
- **Query Processor** — interprets SQL commands into actionable tasks
- **DDL (Data Definition Language)** — defines schema (CREATE, ALTER)
- **DML (Data Manipulation Language)** — INSERT, UPDATE, DELETE
- **Transaction Manager** — ensures integrity/consistency during concurrent access

Types of DBMS

- **Relational (RDBMS)** — tables with rows/columns, strict schema, SQL. E.g., MySQL, PostgreSQL
- **NoSQL** — unstructured/semi-structured data, document/key-value/graph models. E.g., MongoDB, Redis
- **Hierarchical** — tree structure. E.g., IBM's IMS
- **Network** — graph with multiple parent-child relationships. E.g., IDS
- **Object-Oriented (OODBMS)** — object-oriented data models. E.g., ObjectDB

Advantages of Using DBMS

- **Data Centralization** — single location, reduced redundancy
- **Data Integrity and Consistency** — enforces accuracy rules
- **Data Security** — authentication and access controls
- **Backup and Recovery** — data protected, restorable on failure
- **Concurrent Access** — multiple users, no conflicts

Real-World Applications

- Banking — accounts, transactions, loans
- Healthcare — patient records, prescriptions, billing
- Retail — inventory, orders, sales analytics
- Education — student management, registrations, grading
- Social Media — profiles, posts, interactions

Evolution of Database Systems

1. Early Systems — flat-file, manual storage
2. Hierarchical/Network Models — 1960s, complex relationships
3. Relational Databases — introduced by E.F. Codd (1970s), became the standard
4. NoSQL/Cloud Databases — 21st century, big data & distributed systems

WEEK 2: Data Models

Introduction to Data Models

Definition: Defines the structure, organization, and constraints of data stored in a database — a framework for how data is stored, accessed, and manipulated.

Purpose: represent real-world entities/relationships, ensure consistency/integrity, facilitate database design.

Components: Structure (organization), Operations (insert/update methods), Constraints (integrity rules).

1. Hierarchical Data Model

- **Definition:** Tree structure with parent-child relationships
- **Characteristics:** each child has only one parent; hierarchy-based relationships
- **Advantages:** simple, fast access for parent-child relationships
- **Disadvantages:** rigid, difficult many-to-many relationships, restructuring needed for changes
- **Example:** Organizational chart (each employee reports to one manager); Electronics → Televisions/Portable Electronics → sub-items

2. Network Data Model

- **Definition:** Graph structure, allows multiple parent-child relationships
- **Characteristics:** elements connected by links; more flexible than hierarchical
- **Advantages:** supports many-to-many relationships, efficient for complex relationships
- **Disadvantages:** complex to design/maintain, needs specialized knowledge
- **Example:** Flight reservation system (a flight connects to multiple airports)

3. Relational Data Model

- **Definition:** Data organized in tables (relations) with rows and columns
- **Characteristics:** each table = entity, each row = instance; relationships via primary/foreign keys
- **Advantages:** simple, widely used, SQL support, ensures consistency/integrity
- **Disadvantages:** performance issues with very large datasets, limited for complex hierarchical/network relationships
- **Example:** Student database with Students, Courses, Enrollments tables

4. Object-Oriented Data Model

- **Definition:** Combines OOP concepts with database principles
- **Characteristics:** data as objects with attributes/methods; supports inheritance, encapsulation, polymorphism
- **Advantages:** ideal for complex data relationships, supports multimedia
- **Disadvantages:** complex to implement/maintain, less popular than relational
- **Example:** Multimedia library storing books, videos, audio as objects

5. Entity-Relationship (ER) Model

- **Definition:** Represents data as entities, attributes, and relationships
- **Characteristics:** Entities (Student, Course), Attributes (Name, Age), Relationships (Enrolled)
- **Advantages:** easy to visualize via ER diagrams, useful for relational DB design
- **Disadvantages:** limited to high-level design, needs conversion to physical model
- **Example:** ER diagram showing Students enrolled in Courses taught by Professors

6. NoSQL Data Models

- **Definition:** designed for unstructured/semi-structured data, high scalability/flexibility
- **Types:** Document (MongoDB), Key-Value (Redis), Column-Family (Cassandra), Graph (Neo4j)
- **Advantages:** highly scalable for big data, flexible schema
- **Disadvantages:** lacks standardization, limited complex-query support
- **Example:** Social media platforms storing user interactions in a graph database

Comparison of Data Models

Model	Structure	Use Case	Example
Hierarchical	Tree	Organizational chart	IBM IMS
Network	Graph	Flight reservation system	IDS
Relational	Tables	Enterprise applications	MySQL, Oracle
Object-Oriented	Objects	Multimedia/CAD applications	ObjectDB
ER Model	Entities and Relations	Database design	ER Diagrams
NoSQL	Document, Key-Value	Big data, real-time analytics	MongoDB, Cassandra

WEEK 3: Database Design & the Entity-Relationship Model

Introduction to Database Design

Definition: Process of defining structure, storage, and organization of data to ensure efficiency, scalability, and integrity.

Objectives: minimize redundancy, ensure consistency/integrity, enhance retrieval/manipulation efficiency.

Phases of Database Design

1. Conceptual Design

- High-level representation using ER diagrams
- Key components: Entities (Customer, Product), Attributes (Customer Name, Product Price), Relationships (Customer purchases Product)
- Tools: ER Diagrams, UML Diagrams
- Example: ER diagram of Customers placing multiple Orders

2. Logical Design

- Converts conceptual design into a DBMS-implementable logical model
- Steps: transform entities into tables → define primary keys → define relationships via foreign keys → normalize

Normalization (organizing data to minimize redundancy and improve integrity):

- **1NF** — eliminate repeating groups, ensure atomic values
- **2NF** — eliminate partial dependencies (non-key attributes depend on the whole primary key)
- **3NF** — eliminate transitive dependencies (non-key attributes depend only on the primary key)

3. Physical Design

- Focuses on physical storage and performance optimization
- Key considerations: Indexing (faster retrieval), Partitioning (splitting large tables), Storage Optimization
- Example: index on "CustomerID" to speed up searches

Steps in Database Design

1. Requirement Analysis — gather stakeholder requirements
2. Conceptual Modeling — ER diagrams for entities/attributes/relationships
3. Logical Modeling — convert to relational schema, normalize
4. Physical Modeling — define disk storage, optimize performance
5. Implementation — create schema in a DBMS, populate data
6. Testing and Validation — verify requirements & performance

Key Principles of Good Database Design

- **Simplicity** — as simple as possible while meeting requirements
- **Scalability** — handle growing data
- **Integrity** — enforced via constraints/normalization
- **Flexibility** — adaptable to future changes

- **Performance** — optimized for query speed/storage

Challenges in Database Design

- Balancing normalization vs performance
- Handling complex relationships and large datasets
- Security and access control
- Accommodating changing requirements

Tools for Database Design

- ERD Tools: Lucidchart, draw.io, Microsoft Visio
- DBMS Tools: MySQL Workbench, SSMS, Oracle SQL Developer
- Other: dbdiagram.io, DbSchema, Navicat

Entity-Relationship (ER) Model — Detailed

ERD Definition: A visual representation of different data using conventions that describe how the data relate to each other.

- Rectangles = **entities**
- Diamonds = **relationships**

Entity

- A real-world object (animate or inanimate) that is easily identifiable
- Example: in a school database — students, teachers, classes, courses offered
- **Entity set** — a collection of similar types of entities (e.g., all students of a school)

Weak Entity

- An entity that depends on the existence of another entity; cannot be identified by its own attributes alone
- Uses a foreign key combined with its own attributes to form a primary key
- Example: **Order Item** is meaningless without an **Order**

Attributes

- Properties of entities, represented by ellipses, directly connected to their entity
- All attributes have values within a defined domain/range (e.g., a name can't be numeric; age can't be negative)

Types of Attributes:

- **Simple** — atomic, cannot be divided further (e.g., phone number)
- **Composite** — made of more than one simple attribute (e.g., Name = First_name + Last_name) — shown as a tree of ellipses
- **Derived** — not stored directly; calculated from other attributes (e.g., Age from Date_of_Birth; Area from Radius) — shown as a **dashed ellipse**
- **Single-value** — contains one value (e.g., Social_Security_Number)
- **Multi-value** — may contain more than one value (e.g., multiple phone numbers) — shown as a **double ellipse**

- These combine as: simple single-valued, simple multi-valued, composite single-valued, composite multi-valued

Entity-Set and Keys

- **Key** — an attribute (or set) that uniquely identifies an entity among an entity set (e.g., roll_number for students)
- **Super Key** — a set of attributes that collectively identify an entity
- **Candidate Key** — a minimal super key; an entity set may have more than one
- **Primary Key** — the candidate key chosen by the designer to uniquely identify the entity set

Relationship

- The association among entities (e.g., Employee **works_at** Department; Student **enrolls** in Course)
- Represented by a diamond-shaped box, labeled with a verb, connected to participating entities by lines

Degree of Relationship (number of participating entities):

- Binary = degree 2
- Ternary = degree 3
- n-ary = degree n

Mapping Cardinalities (number of entities in one set associated via relationship with the other set):

- **One-to-one** — one entity in A associates with at most one in B, and vice versa
- **One-to-many** — one entity in A associates with many in B, but each B entity associates with at most one A
- **Many-to-one** — many entities in A associate with at most one in B
- **Many-to-many** — entities in A can associate with multiple in B and vice versa

Participation Constraints:

- **Total Participation** — every entity is involved in the relationship (shown with double lines)
- **Partial Participation** — not all entities are involved (shown with single lines)

Generalization, Specialization, and Inheritance

- **Generalization** — going up the hierarchy; entities are combined based on shared characteristics into a more general entity. Example: Pigeon, Sparrow, Dove → generalized as **Birds**
- **Specialization** — the opposite; a group of entities is divided into sub-groups based on characteristics. Example: Person → Student, Teacher (via an "IS A" relationship)
- **Inheritance** — lower-level entities inherit attributes of higher-level entities. Example: Student and Teacher inherit Name, Age, Gender from Person; Student adds Roll_No, Teacher adds Emp_ID
- The process of hiding entity details from the user is called **abstraction**

Worked ER Diagram Examples

Example 1: National Hockey League (NHL) Database

Requirements:

- NHL has many teams; each team has a name, city, coach, captain, and set of players
- Each player belongs to only one team; has a name, position, skill level, and set of injury records

- A team captain is also a player
- A game is played between two teams (host_team, guest_team) with a date and score

ER Structure:

- Entities: **team** (t-name, city, coach), **player** (p-name, position, skill level), **injury record** [weak entity] (id, description)
- Relationships: **belongs_to** (player→team), **captain** (player→team, since captain IS a player), **log** (player→injury record), **game** (team↔team, host/guest roles, with date & score attributes)

Example 2: University Registrar's Office

Requirements:

- Courses (number, title, credits, syllabus, prerequisites)
- Course offerings (course number, year, semester, section number, instructor(s), timings, classroom)
- Students (student-id, name, program)
- Instructors (identification number, name, department, title)
- Enrollment of students in courses + grades awarded

ER Structure:

- Entities: **student** (sid, name, program), **course-offerings** [weak entity] (secno, time, room, year, semester), **instructor** (iid, name, dept, title), **course** (courseno, title, credits, syllabus)
- Relationships: **enrols** (student↔course-offerings, with grade attribute), **teaches** (course-offerings↔instructor), **is_offered** (course-offerings→course), **requires** (course↔course, for prerequisite/maincourse — a self-relationship)

Example 3: Car Insurance Company

Requirements:

- Customers own one or more cars each
- Each car has zero to any number of recorded accidents

ER Structure:

- Entities: **person** (driver-id, name, address), **car** (license, year, model), **accident** (report-number, date, location)
- Relationships: **owns** (person↔car), **participated** (person↔car↔accident, with damage-amount attribute)

Resulting Relational Tables:

```
person (driver-id, name, address)
car (license, year, model)
accident (report-number, date, location)
participated (driver-id, license, report-number, damage-amount)
```